08. Relayer 앱 구조 및 초기화 분석

개요

Relayer는 Omni 네트워크에서 크로스체인 메시지를 감지하고 목적지 체인으로 전송하는 핵심 서비스입니다. 이 문서는 Relayer의 앱 구조와 초기화 과정을 상세히 분석합니다.

전체 아키텍처

디렉토리 구조

```
relayer/
├── main.go # 메인 엔트리 포인트
├── cmd/ # CLI 명령어 처리
| ├── cmd.go # 루트 명령어 정의
| └── flags.go # 플래그 바인딩
├── app/ # 핵심 애플리케이션 로직
| ├── app.go # 메인 애플리케이션 실행
| ├── config.go # 설정 관리
| ├── bootstrap.go # 부트스트랩 오프셋
| ├── worker.go # 워커 로직
| ├── sender.go # 트랜잭션 전송
| ├── creator.go # 서브미션 생성
```

```
| ├── cursors.go # 커서 관리
| ├── buffer.go # 비동기 버퍼
| ├── monitor.go # 모니터링 서버
| ├── metrics.go # 메트릭 정의
| └── types.go # 타입 정의
└── Dockerfile # 컨테이너 빌드
```

메인 엔트리 포인트

main.go

```go
func main() {
libcmd.Main(appcmd.New())
}
```

Relayer는 `lib/cmd` 패키지의 공통 CLI 프레임워크를 사용하여 시작됩니다.

CLI 명령어 구조

cmd.go - 루트 명령어 정의

```go
func New() *cobra.Command {
cmd := libcmd.NewRootCmd(
"relayer",
"Relayer is a service that relays txs between the omni network and rollups",
```

```
buildinfo.NewVersionCmd(),
)

cfg := relayer.DefaultConfig()
bindRunFlags(cmd.Flags(), &cfg)

logCfg := log.DefaultConfig()
log.BindFlags(cmd.Flags(), &logCfg)

cmd.RunE = func(cmd *cobra.Command, _ []string) error {
ctx, err := log.Init(cmd.Context(), logCfg)
if err != nil {
return err
}

if err := libcmd.LogFlags(ctx, cmd.Flags()); err != nil {
return err
}

return relayer.Run(ctx, cfg)
}

return cmd
```

```
}
```

주요 기능:

- Cobra 기반 CLI 구조
- 설정 및 로그 플래그 바인딩
- `relayer.Run()` 함수로 메인 로직 실행

flags.go - 플래그 바인딩

```
func  bindRunFlags(flags *pflag.FlagSet, cfg *relayer.Config) {
netconf.BindFlag(flags, &cfg.Network)
xchain.BindFlags(flags, &cfg.RPCEndpoints)
flags.StringVar(&cfg.PrivateKey, "private-key", cfg.PrivateKey, "The path to the
flags.StringVar(&cfg.HaloCometURL, "halo-url", cfg.HaloCometURL, "The URL of the
flags.StringVar(&cfg.HaloGRPCURL, "halo-grpc-url", cfg.HaloGRPCURL, "The gRPC UR
flags.StringVar(&cfg.MonitoringAddr, "monitoring-addr", cfg.MonitoringAddr, "The
flags.StringVar(&cfg.DBDir, "db-dir", cfg.DBDir, "The path to the database direc
flags.StringVar(&cfg.CoinGeckoAPIKey, "coingecko-apikey", cfg.CoinGeckoAPIKey, "
}
```

설정 가능한 플래그:

- `--network` : 네트워크 ID (devnet, testnet, mainnet)
- `--private-key` : 릴레이어 개인키 경로
- `--halo-url` : Halo 노드 CometBFT URL

- `--halo-grpc-url` : Halo 노드 gRPC URL (선택사항, 더 빠름)
- `--monitoring-addr` : 모니터링 서버 주소
- `--db-dir` : 데이터베이스 디렉토리
- `--coingecko-apikey` : 토큰 가격 조회용 API 키

설정 관리

Config 구조체

```
type Config struct {
RPCEndpoints xchain.RPCEndpoints
PrivateKey  string
HaloCometURL  string
HaloGRPCURL  string
Network netconf.ID
MonitoringAddr  string
DBDir  string
CoinGeckoAPIKey  string
}

func  DefaultConfig() Config {
return Config{
PrivateKey: "relayer.key",
HaloCometURL: "localhost:26657",
```

```
HaloGRPCURL: "",
Network: "",
MonitoringAddr: ":26660",
DBDir: "./db",
}
}
```

설정 템플릿:

- TOML 템플릿 파일을 통한 설정 파일 생성
- `WriteConfigTOML()` 함수로 설정 파일 작성

메인 애플리케이션 실행

app.go - Run 함수

```
func  Run(ctx context.Context, cfg Config) error {
log.Info(ctx, "Starting relayer")

buildinfo.Instrument(ctx)
ctx = chaos.WithErrProbability(ctx, cfg.Network)

// 1. 모니터링 서버 시작
monitorChan := serveMonitoring(cfg.MonitoringAddr)
```

```go
// 2. Portal Registry 초기화
portalReg, err := makePortalRegistry(ctx, cfg.Network, cfg.RPCEndpoints)
if err != nil {
return err
}

// 3. 네트워크 설정 대기
network, err := netconf.AwaitOnExecutionChain(ctx, cfg.Network, portalReg, solve
if err != nil {
return err
}

// 4. RPC 클라이언트 초기화
rpcClientPerChain, err := initializeRPCClients(ctx, network.EVMChains(), cfg.RPC
if err != nil {
return err
}

// 5. 개인키 로드
privateKey, err := ethcrypto.LoadECDSA(cfg.PrivateKey)
if err != nil {
return errors.Wrap(err, "failed to load private key")
}
```

```go
// 6. CChain Provider 초기화
cprov, err := newCProvider(ctx, cfg)
if err != nil {
return err
}

// 7. XChain Provider 초기화
xprov := xprovider.New(network, rpcClientPerChain, cprov)

// 8. 토큰 가격 조회 및 PnL 로거 초기화
pricer := newTokenPricer(ctx, cfg.CoinGeckoAPIKey)
pnl := newPnlLogger(network.ID, pricer)

// 9. 데이터베이스 및 커서 초기화
db, err := initializeDB(ctx, cfg)
if err != nil {
return err
}
cursors, err := cursor.New(db, xprov.GetSubmittedCursor, network)
if err != nil {
return err
}
```

```go
cursors.StartLoops(ctx)


// 10. 각 목적지 체인별 워커 시작
for _, destChain := range network.EVMChains() {
// Sender 설정
sendProvider := func() (SendAsync, error) {
sender, err := NewSender(
network.ID,
destChain,
rpcClientPerChain[destChain.ID],
*privateKey,
network.ChainVersionNames(),
pnl.log,
)
if err != nil {
return nil, err
}


return sender.SendAsync, nil
}


// Validator Set Awaiter 설정
portal, err := bindings.NewOmniPortal(destChain.PortalAddress, rpcClientPerChain
```

```go
if err != nil {
return errors.Wrap(err, "create portal contract")
}
awaitValSet := newValSetAwaiter(portal, destChain.BlockPeriod)

// 워커 시작
worker := NewWorker(
destChain,
network,
cprov,
xprov,
CreateSubmissions,
sendProvider,
awaitValSet,
cursors,
)

go worker.Run(ctx)
}

// 11. 종료 대기
select {
case <-ctx.Done():
```

```
log.Info(ctx, "Shutdown detected, stopping...")
return nil
case err := <-monitorChan:
return err
}
}
```

핵심 컴포넌트 초기화

1. CChain Provider 초기화

```
func newCProvider(ctx context.Context, cfg Config) (cchain.Provider, error) {
if cfg.HaloGRPCURL != "" {
log.Debug(ctx, "Using grpc cprovider", "url", cfg.HaloGRPCURL)

encConf, err := haloapp.ClientEncodingConfig(ctx, cfg.Network)
if err != nil {
return nil, errors.Wrap(err, "client encoding config")
}

return cprovider.NewGRPC(cfg.HaloGRPCURL, cfg.Network, encConf.InterfaceRegistr
}
```

```
log.Debug(ctx, "Using comet cprovider", "url", cfg.HaloCometURL)


c, err := http.New("tcp://"+cfg.HaloCometURL, "/websocket")
if err != nil {
return nil, errors.Wrap(err, "new tendermint client")
}


return cprovider.NewABCI(c, cfg.Network), nil
}
```

Provider 선택 로직:

- gRPC URL이 설정된 경우: gRPC Provider 사용 (더 빠름)
- 그렇지 않은 경우: ABCI Provider 사용 (CometBFT WebSocket)

2. RPC 클라이언트 초기화

```
func initializeRPCClients(ctx context.Context, chains []netconf.Chain, endpoint
rpcClientPerChain := make(map[uint64]ethclient.Client)
for _, chain := range chains {
rpc, err := endpoints.ByNameOrID(chain.Name, chain.ID)
if err != nil {
return nil, err
}
c, err := ethclient.DialContext(ctx, chain.Name, rpc)
```

```go
if err != nil {
return nil, errors.Wrap(err, "dial rpc", "chain_name", chain.Name, "chain_id",
}
go c.CloseIdleConnectionsForever(ctx)

rpcClientPerChain[chain.ID] = c
}

return rpcClientPerChain, nil
}
```

기능:

- 각 체인별 RPC 클라이언트 생성
- 유휴 연결 자동 정리 고루틴 시작

3. 데이터베이스 초기화

```go
func initializeDB(ctx context.Context, cfg Config) (dbm.DB, error) {
if cfg.DBDir == "" {
log.Warn(ctx, "No --db-dir provided, using in-memory DB", nil)
return dbm.NewMemDB(), nil
}
```

```go
db, err := dbm.NewGoLevelDB("app", cfg.DBDir, nil)
if err != nil {
return nil, errors.Wrap(err, "new golevel db")
}


return db, nil
}
```

데이터베이스 옵션:

- 디렉토리 지정시: LevelDB 사용
- 디렉토리 미지정시: 인메모리 DB 사용

부트스트랩 시스템

bootstrap.go - 오프셋 관리

```go
func getBootstrapOffset(network netconf.Network, destChain netconf.Chain, chain
chainVerName := network.ChainVersionName(chainVer)
for _, b := range bootstrapCursors[network.ID] {
if b.Worker == destChain.Name && b.ChainVersion == chainVerName {
return b.AttestOffset, true
}
}
```

```
return 0, false
}

type bootstrapCursor struct {
ChainVersion string
Worker string
AttestOffset uint64
}
```

부트스트랩 커서:

- 네트워크별 사전 정의된 오프셋
- Grafana에서 수집된 실제 운영 데이터 기반
- 새로운 릴레이어 인스턴스의 빠른 동기화 지원

모니터링 시스템

monitor.go - 모니터링 서버

```
func serveMonitoring(address string) <-chan error {
errChan := make(chan error)
go func() {
mux := http.NewServeMux()
mux.Handle("/metrics", promhttp.Handler())
```

```
// pprof 엔드포인트
mux.HandleFunc("/debug/pprof/", pprof.Index)
mux.HandleFunc("/debug/pprof/cmdline", pprof.Cmdline)
mux.HandleFunc("/debug/pprof/profile", pprof.Profile)
mux.HandleFunc("/debug/pprof/symbol", pprof.Symbol)
mux.HandleFunc("/debug/pprof/trace", pprof.Trace)

srv := &http.Server{
Addr: address,
ReadHeaderTimeout: 5 * time.Second,
IdleTimeout: 5 * time.Second,
WriteTimeout: 5 * time.Second,
Handler: mux,
}
errChan <- errors.Wrap(srv.ListenAndServe(), "serve monitoring")
}()

return  errChan
}
```

제공 엔드포인트:

- `/metrics` : Prometheus 메트릭
- `/debug/pprof/*` : Go 프로파일링 도구

의존성 주입 패턴

Relayer는 다음과 같은 의존성 주입 패턴을 사용합니다:

1. Provider 패턴

- `CChain Provider` : 합의 체인 데이터 제공
- `XChain Provider` : 크로스체인 데이터 제공
- `Send Provider` : 트랜잭션 전송 기능 제공

2. Factory 패턴

- `CreateFunc` : 서브미션 생성 함수
- `SendAsync` : 비동기 전송 함수
- `awaitValSet` : 검증자 세트 대기 함수

3. 설정 기반 초기화

- 설정에 따른 조건부 컴포넌트 초기화
- 환경별 다른 Provider 선택

에러 처리 및 복구

1. Chaos Engineering

```
ctx = chaos.WithErrProbability(ctx, cfg.Network)
```

2. 지수 백오프

- 연결 실패시 자동 재시도
- 네트워크 지연 대응

3. 그레이스풀 셧다운

```
select {
case <-ctx.Done():
log.Info(ctx, "Shutdown detected, stopping...")
return nil
case err := <-monitorChan:
return err
}
```

요약

Relayer의 앱 구조는 다음과 같은 특징을 가집니다:

1. **모듈화된 설계**: CLI, 설정, 핵심 로직이 분리
2. **의존성 주입**: 테스트 가능하고 유연한 구조
3. **비동기 처리**: 각 체인별 독립적인 워커
4. **모니터링 통합**: Prometheus 메트릭과 pprof 지원
5. **에러 복구**: 자동 재시도와 그레이스풀 셧다운
6. **설정 기반**: 환경별 다른 설정 지원

이러한 구조를 통해 Relayer는 안정적이고 확장 가능한 크로스체인 메시지 릴레이 서비스를 제공합니다.